Adding `operator<=>` to types that are not currently comparable

Document Number: P1191R0
Date: 2018-08-22
Author: David Stone (davidmstone@google.com, david@doublewise.net)
Audience: Library Evolution Working Group (LEWG)

Summary

The following types do not currently have comparison operators. They should be modified as follows:

- `filesystem::file_status`: `strong_equality`
- `filesystem::space_info`: `strong_equality`
- `slice`: `strong_equality`
- `to_chars_result`: `strong_equality`
- `from_chars_result`: `strong_equality`

Discussion

`filesystem::file_status` is conceptually a struct with two enum data members (but with a get / set function interface). It is equal if both the `type` and the `permissions` compare equal.

`filesystem::space_info` is a struct with three `uintmax_t` data members: `capacity`, `free`, and `available`. Two `space_info` objects compare equal if all data members compare equal.

`slice` is conceptually a struct with three `size_t` data members (but with getters only): `start()`, `size()`, and `stride()`. Two `slice` objects compare equal if all three values are equal.

This paper does not propose adding `operator<=>` to `gslice`. This object is much like `slice` except `size` and `stride` are instances of `valarray<size_t>` rather than just `size_t`. Since `valarray` does not have a traditional comparison, we do not attempt to define the equivalent for `gslice`.

`to_chars_result` is a struct that stores an iterator `ptr` and an `errc` `ec`. Two `to_chars_result` objects compare equal if both `ptr` and `ec` compare equal.

`from_chars_result` is a struct that stores an iterator `ptr` and an `errc` `ec`. Two `from_chars_result` objects compare equal if both `ptr` and `ec` compare equal.